Systems & Control Transactions

Original Research Article
Peer Reviewed Conference Proceeding
FOCAPO-CPC 2027
Tucson, Arizona, USA, 10-14 January 2027

PSE PRESS

Challenges of Gate-Based Quantum Optimization for a Discrete Process-Design Problem

Yirang Park[a] and David E. Bernal Neira[a]

[a] Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

ABSTRACT

Discrete decisions, such as selecting routes, units, and operating modes, are an early and combinatorially hard step in process design. Gate-based quantum optimization algorithms, namely the quantum approximate optimization algorithm (QAOA) and the variational quantum eigensolver (VQE), are increasingly proposed for such problems, which raises a practical question for the process systems engineering community. Once a discrete-design model is mapped to the quadratic unconstrained binary optimization (QUBO) form required by these methods, do the resulting samples actually correspond to process-design decisions, and how do they compare with classical methods? We study this question on a small but fully enumerable drug-substance manufacturing instance with 19 binary flow decisions. The penalty reformulation needed to obtain a QUBO adds 17 auxiliary variables, so a sampled 36-bit state is not itself a design decision: it becomes one only after projection onto the 19 flow variables and exact repair of the auxiliaries. After this preprocessing, energy-targeted transferred-angle QAOA concentrates probability on the optimal flowsheet (664 of 262144 ideal reads), but does not outperform a simple classical hill climber, an exact solver, or exhaustive enumeration. A small IBM hardware run completes the same pipeline without adding an advantage. We report these results as a workflow-feasibility and interpretation study, not as a claim of quantum speedup.

Keywords: quantum computing, process design, discrete optimization, QUBO reformulation, pharmaceutical manufacturing

INTRODUCTION

Discrete choices are often the first barrier in process design. Before a flowsheet can be simulated or optimized over continuous variables, the designer must commit to a set of mutually exclusive options: which synthesis route to use, which unit operations to include, and which operating modes to run [1]. In pharmaceutical manufacturing, these choices are particularly consequential because route and equipment selection shape cost, yield, and regulatory burden, and because digital design and optimization tools for this sector are comparatively recent [2, 3]. The resulting models are combinatorial, and even modestly sized superstructures generate large discrete search spaces.

Gate-based quantum optimization is one of the emerging tools proposed for this setting. A gate-based quantum computer operates on quantum bits (qubits) by applying sequences of parameterized gate operations that form a quantum circuit; measuring the circuit returns a binary string, and the circuit parameters—called angles—are tuned so that low-energy strings appear with high probability. The quantum approximate optimization algorithm (QAOA) [4] and the variational quantum eigensolver (VQE) [5, 6] follow this variational paradigm: QAOA uses p alternating cost and mixer layers, and VQE uses a hardware-efficient parameterized ansatz; in both cases, a classical optimizer adjusts the angles. Current hardware is noisy and of intermediate scale [7], and benchmarking surveys stress that a practical quantum advantage for optimization has not yet been established [8]; the relevant question is what is required to use these methods today and how their output compares with classical baselines.

Within process systems engineering, quantum computing has been reviewed as an emerging capability [9], and hybrid quantum-classical strategies have been proposed for discrete optimization in process and energy systems [10]. Discrete-design instances have been mapped to the QUBO form [11] and solved with Ising-based annealing hardware [10, 12]. These works establish that the mapping is possible; few examine what happens to the samples after the reformulation, and the gate-based path is less explored than annealing. Casting a constrained problem as a QUBO introduces auxiliary variables [13] that are bookkeeping rather than design decisions, so a raw quantum sample is not directly interpretable as a flowsheet. The question is: what does it take to turn gate-based quantum samples into process-design decisions, and how does this compare with classical methods?

We answer this through an end-to-end audit on a single drug-substance manufacturing (DS-MFG) instance, chosen to be small enough to enumerate exactly yet large enough to expose the reformulation and interpretation issues that arise at scale. Our contribution is not a new QAOA or VQE variant. We find: (i) raw QUBO samples require projection and exact auxiliary repair before they are interpretable as design candidates; (ii) a quadratic surrogate over the design variables fits the repaired landscape globally but fails as an optimizer; (iii) transferred-angle QAOA concentrates probability on good flowsheets but does not beat classical baselines; and (iv) a hardware run confirms pipeline feasibility without adding advantage.

For the process systems engineering community, the value of this study is a concrete, reproducible measurement of the engineering burden that sits between a discrete-design model and a usable quantum sample, on a problem from a relevant application domain. The conclusion is deliberately conservative: the results demonstrate feasibility and interpretation, not a runtime or hardware advantage over classical computation.

METHODS

The Discrete Design Instance

The DS-MFG instance is the discrete subproblem of a drug-substance manufacturing case study introduced by Park and Bernal Neira [12], where a mixed-integer nonlinear design problem is decomposed so that the discrete part is handled by combinatorial solvers and the nonlinear part by simulators. We keep the same discrete instance but ask what is required to run gate-based optimizers. The instance is a binary integer program for capital-expenditure selection in the manufacturing superstructure: the 19 binary variables encode candidate material-flow arcs and unit-operation selections, and the objective function is a normalized capital-cost score. The discrete problem has the form

$$
\begin{aligned}
\min_{f} \quad & \sum_{i \in F} c_i f_i \\
\text{s.t.} \quad & \sum_{i \in \mathrm{In}(n)} f_i = \sum_{j \in \mathrm{Out}(n)} f_j \qquad && \forall n \in N, \\
& f_{\text{source}} = 1, \quad f_{\text{sink}} = 1, \\
& g(f) \le 0, \\
& f_i \in \{0,1\} && \forall i \in F.
\end{aligned} \tag{1}
$$

Equation (1) lists the relevant constraint classes: flow balance at each node $n \in N$, fixed source and sink selections, and additional logical constraints $g(f) \le 0$ that enforce a valid flowsheet.

Solved with `Gurobi` [14], the model (19 binary variables, 20 constraints) returns an optimal capital-cost score of 11.7095; with pool enumeration enabled (`PoolSearchMode=2`), all 36 feasible flows are retained in a single deterministic solve (0.073 s). We treat 11.7095 as the certified global optimum and use it as the reference throughout.

From Integer Program to QUBO and Back

To run QAOA and VQE, the integer program must be expressed as a QUBO, in which all constraints are folded into a single quadratic objective over binary variables [11, 13]. Inequality constraints require auxiliary (slack) variables; the penalty weights and auxiliary count are not unique, and both affect how faithfully the QUBO represents the original problem [13]. For this instance, the penalty reformulation introduces 17 auxiliary variables, so the QUBO acts on 36 binary variables. For a bitstring $x \in \{0,1\}^{36}$, the encoded objective is

$$
E_{\text{QUBO}}(x) = \ell^{\top} x + x^{\top} Q x + b, \tag{2}
$$

with linear coefficients ℓ, quadratic coefficient matrix Q, and constant offset b.

The 36 variables are separated into the original flow variables and the auxiliaries,

$$
x = (y, z), \qquad y \in \{0,1\}^{19}, \quad z \in \{0,1\}^{17}.
$$

Only y encodes a design decision; z is bookkeeping. A sampled 36-bit string does not map directly to a flowsheet.

We make it interpretable by projecting onto y and repairing the auxiliaries, choosing the assignment that minimizes the QUBO energy for that flow,

$$F(y) = \min_{z\in\{0,1\}^{17}} E_{\text{QUBO}}(y,z). \tag{3}$$

The repair is exact and cheap here because, once y is fixed, the auxiliary-auxiliary coupling graph splits into 13 small connected components solved by enumeration. For a general QUBO, the auxiliary repair is itself NP-hard, a limitation we return to in the Discussion.

The repaired minimizer over all 2^{19} flows recovers the optimum, and the repaired QUBO scores match the integer-program objective to within 3.1×10^{-13}, confirming they encode the same objective. Exact enumeration finds exactly 36 feasible flows, as does the solution pool option of `Gurobi`. Note that 14 of the 50 lowest-cost repaired flows are infeasible flows whose penalty values are insufficient to place them above all feasible ones.

The repaired objective $F(y)$ is not a quadratic function of y, but QAOA and VQE require a QUBO input. Rather than using the full 36-variable QUBO (which mixes design variables with auxiliaries and requires 36-qubit circuits), we construct a reduced, 19-variable QUBO directly over the design space y by fitting a quadratic surrogate to the repaired landscape:

$$\hat{F}(y) = \hat{b} + \hat{\ell}^{\top}y + y^{\top}\hat{Q}y \tag{4}$$

by least squares over all 2^{19} repaired flows. The surrogate fits the landscape almost perfectly in aggregate ($R^2 \approx 0.999$), but this global accuracy is misleading for optimization: the surrogate's own minimizer repairs to 14.5505 rather than the true 11.7095, the true optimum ranks only 33rd under the surrogate, and among the 50 lowest-cost repaired flows the surrogate's absolute errors reach 29.4. We therefore use $\hat{F}$ only to generate samples via the quantum algorithms; every reported sample-quality metric is computed with the exact repaired objective F, not $\hat{F}$.

Quantum, Classical, and Hardware Workflows

We evaluate several workflows over this pipeline. The local QAOA and VQE workflows run through `QiskitOpt.jl`, a Julia wrapper for Qiskit [15], with QUBO models constructed via the `QUBO.jl` ecosystem [13]. In QAOA, we varied the parameter of repetition of mixer and driving Hamiltonians, p, which, as it tends to infinity, converges to the adiabatic quantum algorithm [16] with convergence guarantees yet yields deeper quantum circuits, more prone to error in current NISQ devices [7]. Both algorithms have a number of other parameters, also called angles, that are found variationally by an external classical optimizer [6], in our case COBYLA. Direct full-QUBO QAOA and VQE operate on the 36-variable formulation; reduced-surrogate QAOA and VQE operate on the 19-variable surrogate and are scored by exact repair. The best-performing quantum workflow adds an offline parameter-search step: QAOA parameters are optimized on the surrogate with JuliQAOA [17] by minimizing the expected surrogate energy $\mathbb{E}_{y\sim p_\theta}[\hat{F}(y)]$, then transferred to a Qiskit circuit for sampling (energy-targeted, non-oracle). The fixed, pre-optimized parameters with the highest local solution concentration are also submitted to the hardware for a feasibility test. Reduced-surrogate VQE uses an `EfficientSU2` ansatz optimized with COBYLA.

As classical baselines, we sample the repaired 19-flow objective by uniform random sampling and by single-bit-flip hill climbing with random restarts. As a hardware feasibility test, we run the highest-concentration fixed-parameter QAOA circuit on the IBM `ibm_fez` quantum computer. An independent state-vector calculation predicts the central QAOA reference-optimum probability at 2.63×10^{-3}, consistent with the 2.53×10^{-3} sampled rate. Software versions, random seeds, simulator settings, circuit depths, and job identifiers are recorded in the artifact bundle.

For each sampled state $x = (y,z)$ we record the raw QUBO energy $E_{\text{QUBO}}(y,z)$, the repaired objective $F(y)$, and the auxiliary gap. A *reference-optimum hit* means the repaired flow equals the certified optimal flowsheet (the unique minimum-cost feasible design). A *feasible-design hit* means the repaired flow matches any of the 36 feasible flows. We report both hit rates with two-sided Wilson 95% confidence intervals (score-based intervals for proportions that remain valid near zero).

RESULTS

Repair Changes the Meaning of a Sample

Sampling the 36-variable QUBO directly does not yield an interpretable design result. A $p = 2$ full-QUBO QAOA found no optimum in 512 samples; a 32768-read run returned it only 4 times after repair, and never as a fully optimal 36-bit string without projection. Without projection and exact repair, a raw QUBO sample cannot be read as a flowsheet. Figure 1 summarizes the scoring pipeline the remaining results apply.

Reduction and Parameter Transfer Concentrate QAOA Samples

Once the pipeline is in place, reduction to the surrogate and transferred-parameter search are what make QAOA samples concentrate on feasible, optimal flowsheets. The energy-targeted $p = 5$ transferred workflow sampled

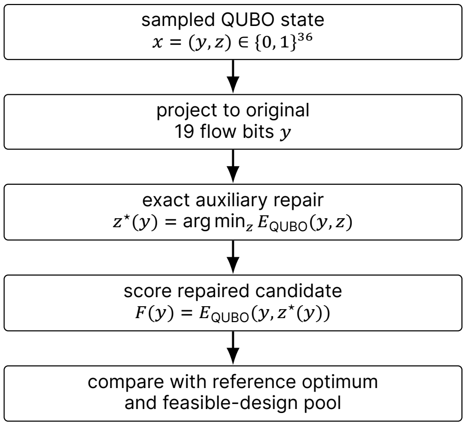

Figure 1. Repair-aware scoring pipeline applied to every sampled QUBO state: raw 36-bit strings are projected to the 19 flow variables, repaired exactly, and scored against the certified optimum and the feasible-design pool. Hit rates for all workflows appear in Table 1.

the certified optimum 664 times in 262144 ideal reads (reference-optimum rate 2.53×10^{-3}, Wilson 95% interval $[2.35, 2.73] \times 10^{-3}$; TTS99-opt 0.37 s, TTS99-feas 0.007 s).

VQE and Classical Baselines Bound the Claim

VQE benefits from the same reduction but produces a flatter distribution. Full-QUBO VQE found no optimum in a five-seed sweep; a selected-seed reduced-surrogate VQE found the optimum 20 times in 524288 reads (TTS99-opt 6.93 s, TTS99-feas 0.36 s). Hill climbing found the optimum 19 times in 262144 evaluations (TTS99-opt 0.23 s, TTS99-feas 0.006 s); random sampling never found the optimum (TTS99-feas 0.46 s). TTS99 accounts for time per sample and hit probability, but excludes offline preprocessing that dominates the real QAOA cost.

Hardware Execution Is an Implementation Feasibility Check

The highest-concentration fixed-angle QAOA circuit ran on the `ibm_fez` quantum computer across three repeats and three random transpile seeds (92001–92003), with 4096 measurement shots per repeat, for 36864 total hardware reads (9 jobs, 73.5 s elapsed including cloud queue). The run produced 4 feasible-design reads and no reference-optimum read; the best hardware sample was the second-cheapest feasible flowsheet (objective 11.8105). The resulting TTS99-feas is 84.6 s—roughly 14000× slower than the hill climber's 0.006 s and 1000× slower than Gurobi's 0.073 s, primarily because most elapsed time is cloud overhead rather than circuit execution (≈2 ms per shot). With zero optimum hits, the Wilson 95% upper bound on the hardware optimum-hit rate is about 1.04×10^{-4}. Because calibration and full transpilation metadata were not retained, this run confirms only that the submission-and-scoring path executes end to end on hardware.

DISCUSSION AND LIMITATIONS

This study answers a narrower question: not whether quantum optimization solves the DS-MFG instance (Gurobi does in under a second), but what must happen for a QAOA or VQE distribution to yield meaningful process-design samples. That engineering path requires variable separation, exact auxiliary repair, a surrogate for the QUBO interface, offline angle search, and scoring in the original flow space. The repaired distribution does concentrate on good flowsheets; it does not translate into a runtime advantage over classical methods for this instance.

The main limitations follow directly. Exact auxiliary repair is tractable here only because the conditioned auxiliary graph decomposes; in general, it is NP-hard, and an approximate repair would change the meaning of every reported metric. Read counts exclude the offline preprocessing that dominates the real cost, and meaningful comparison with classical optimization requires accounting for that total cost [8, 18].

CONCLUSION

For this drug-substance manufacturing case study, QAOA and VQE samples become interpretable as process-design decisions only after the QUBO auxiliary variables are separated from the flow variables and repaired exactly. With that bridge in place, energy-targeted transferred-angle QAOA concentrated probability on the optimal flowsheet (664 of 262144 ideal reads) and returned a feasible flowsheet 32502 times, while a selected-seed VQE follow-up reached the optimum 20 times in 524288 reads. A small IBM hardware run completed the same pipeline and found a feasible but non-optimal flowsheet. None of these outperforms a simple classical hill climber or the exact solver.

The lesson for the process systems engineering community is that the engineering effort between a discrete-design model and a usable quantum sample is substantial and easy to understate, and that honest evaluations of gate-based quantum optimization for process design require scoring in the original decision space and reporting the full preprocessing cost.

Table 1: Primary outcomes, all scored after exact auxiliary repair. Reference-hit rate is the empirical probability of sampling the certified optimal flowsheet. TTS99-opt and TTS99-feas are empirical 99% time-to-solution for the reference optimum and any feasible flow, computed as $\tau\lceil \log(0.01)/\log(1-\hat{p}) \rceil$ where τ is wall time per sample; rows with zero hits show ∞. TTS for local methods uses per-sample Aer or script wall time and excludes offline preprocessing (surrogate fitting, angle search). TTS for IBM uses total elapsed time including cloud job queue (73.5 s for 36864 shots, 9 jobs). Gurobi TTS is its deterministic solve time.

Workflow	Reads	Reference-hit rate	TTS99-opt (s)	TTS99-feas (s)	What it shows
Gurobi solution pool	—	Deterministic	0.073	0.073	Certified optimum 11.7095; `PoolSearchMode=2` retains all 36 feasible flows in one shot.
Uniform random sampling	262 k	0	∞	0.46	Weakest baseline; never finds the optimum; best sampled flow is feasible (11.8105).
Hill-climb random restarts	262 k	7.25×10^{-5}	0.23	0.006	Fastest probabilistic path to both targets.
Energy-targeted QAOA, p=5	262 k	2.53×10^{-3}	0.37	0.007	Primary quantum result; highest hit rate but TTS99-opt worse than hill climb.
Reduced-surrogate VQE, seed 74018	524 k	3.81×10^{-5}	6.93	0.36	Reaches the optimum but much flatter distribution than QAOA.
IBM `ibm_fez` pilot	36.9 k	0	∞	84.6	Implementation feasibility check; 4 feasible reads; TTS includes cloud queue time.

ACKNOWLEDGMENTS

This material is based upon work supported by the Center for Quantum Technologies under the Industry-University Cooperative Research Center Program at the US National Science Foundation under Grant No. 2224960.

DECLARATION OF USE OF AI

The authors used generative AI tools to assist in generating initial code, data analysis workflows, and draft visualizations as described in the relevant sections of the manuscript. All AI-generated outputs were reviewed, tested, and modified by the authors. The authors take full responsibility for the accuracy, validity, and integrity of all results.

AUTHOR IDENTIFIERS

Author ORCIDs:
Park: 0009-0008-6629-3308
Bernal Neira: 0000-0002-8308-5016

REFERENCES

[1] L. Mencarelli, Q. Chen, A. Pagot, and I. E. Grossmann. "A review on superstructure optimization approaches in process system engineering". In: *Computers and Chemical Engineering* 136 (2020), p. 106808. DOI: 10.1016/j.compchemeng.2020.106808.

[2] D. Casas-Orozco, D. Laky, V. Wang, M. Abdi, X. Feng, E. Wood, C. Laird, G. V. Reklaitis, and Z. K. Nagy. "PharmaPy: An object-oriented tool for the development of hybrid pharmaceutical flowsheets". In: *Computers & Chemical Engineering* 153 (2021), p. 107408. DOI: 10.1016/j.compchemeng.2021.107408.

[3] D. Laky, D. Casas-Orozco, C. D. Laird, G. V. Reklaitis, and Z. K. Nagy. "Simulation-Optimization Framework for the Digital Design of Pharmaceutical Processes Using Pyomo and PharmaPy". In: *Industrial & Engineering Chemistry Research* 61.43 (2022), pp. 16128–16140. DOI: 10.1021/acs.iecr.2c01636.

[4] E. Farhi, J. Goldstone, and S. Gutmann. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: 1411.4028 [quant-ph]. URL: https://arxiv.org/abs/1411.4028.

[5] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien. "A variational eigenvalue solver on a photonic quantum processor". In: *Nature Communications* 5 (2014), p. 4213. DOI: 10.1038/ncomms5213. arXiv: 1304.3061 [quant-ph].
[6] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles. "Variational quantum algorithms". In: *Nature Reviews Physics* 3 (2021), pp. 625–644. DOI: 10.1038/s42254-021-00348-9. arXiv: 2012.09265 [quant-ph].
[7] J. Preskill. "Quantum Computing in the NISQ era and beyond". In: *Quantum* 2 (2018), p. 79. DOI: 10.22331/q-2018-08-06-79.
[8] A. Abbas, A. Ambainis, B. Augustino, et al. "Challenges and opportunities in quantum optimization". In: *Nature Reviews Physics* 6.12 (2024), pp. 718–735. DOI: 10.1038/s42254-024-00770-9.
[9] D. E. Bernal, A. Ajagekar, S. M. Harwood, S. T. Stober, D. Trenev, and F. You. "Perspectives of quantum computing for chemical engineering". In: *AIChE Journal* 68.6 (2022), e17651. DOI: 10.1002/aic.17651.
[10] A. Ajagekar and F. You. "Quantum computing for energy systems optimization: Challenges and opportunities". In: *Energy* 179 (2019), pp. 76–89.
[11] F. Glover, G. Kochenberger, and Y. Du. *A Tutorial on Formulating and Using QUBO Models*. 2019. arXiv: 1811.11538 [cs.DS]. URL: https://arxiv.org/abs/1811.11538.
[12] Y. Park and D. E. B. Neira. *Leveraging Classical and Quantum Computing for Process Systems Engineering Applications: Decomposition Algorithm with Ising Solvers for Efficient Discrete Landscape Exploration*. 2026. arXiv: 2603.19520 [math.OC]. URL: https://arxiv.org/abs/2603.19520.
[13] P. M. Xavier, P. Ripper, T. Andrade, J. D. Garcia, N. Maculan, and D. E. Bernal Neira. *Qubo.jl: A julia ecosystem for quadratic unconstrained binary optimization*. 2023. DOI: 10.48550/arXiv.2307.02577. arXiv: 2307.02577 [math.OC]. URL: https://arxiv.org/abs/2307.02577.
[14] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2026. URL: https://www.gurobi.com.
[15] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta. *Quantum computing with Qiskit*. 2024. DOI: 10.48550/arXiv.2405.08810. arXiv: 2405.08810 [quant-ph].
[16] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser. "Quantum Computation by Adiabatic Evolution". In: (2000).
[17] J. Golden, A. Bärtschi, D. O'Malley, E. Pelofske, and S. Eidenbenz. "JuliQAOA: Fast, Flexible QAOA Simulation". In: *SC-W '23: Proceedings of the Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. Association for Computing Machinery, 2023, pp. 1454–1459. DOI: 10.1145/3624062.3624220. arXiv: 2312.06451 [quant-ph].
[18] D. E. Bernal Neira, R. Brown, P. Sathe, F. Wudarski, M. Pavone, E. Rieffel, and D. Venturelli. "Benchmarking the operation of quantum heuristics and ising machines: scoring parameter setting strategies on optimization applications". In: *Quantum Machine Intelligence* 7.2 (2025), p. 86. DOI: 10.1007/s42484-025-00311-2.